

Storage, databases and concurrency

Consolidated Perlmutter experiment report

Manoj Khatri · HPCAI Lab · 04 October 2026

Internal findings and analysis. This document closes the agreed experimental round; it is not a publication manuscript.

The CANOPIE extension now supports repeatable, configuration-driven comparisons of Lustre and memory-backed tmpfs across SQLite native, SQLite under Shifter, PostgreSQL with a Shifter server, and InfluxDB with a Shifter server. Metadata, telemetry and mixed data-type workloads run with multiple client processes on one compute node.

Evidence included	Measured / observations	Warmups
Original four-deployment pilots	240	48
1,024-query SQL follow-up	180	36
Mixed-workload pilots	120	24
SQL allocation-repeat study	540	108
Repeated-performance subtotal	1,080	216
10M-record calibration (single observations)	48	0
System diagnostics (single observations)	48	0

Total: 96 runs and 1,392 trial records. Calibration and diagnostics are not pooled with repeated-performance trials. Earlier integration and separate profiling-overhead experiments are outside this package.

Principal findings

Storage benefit depends on workload and concurrency. Across three allocation blocks, native SQLite metadata-write throughput gains fall from **1.87x** on tmpfs at one client to **1.03x** at 64; the latter 95% interval includes parity. PostgreSQL retains a metadata-write gain of about **1.38x** at 64 clients.

The metadata-only result does not describe every workload. The mixed native SQLite pilot retains a **1.50x** mean write gain at 64 clients. Supported system measurements were collected, but the attempted node-wide perf counters were unavailable under the site access policy.

Design, measurement boundaries and settings

Repeated trials write one million total records in batches of 1,000 and then perform warm range-query reads returning 100 rows per query. Original SQL pilots use 64,000 queries; InfluxDB and later studies use 1,024. Each repeated run has one warmup and five measured repetitions per storage/client condition, with seeded condition ordering.

Startup is already excluded from the performance timers. Workers initialize, report readiness and wait for an event. The controller starts its timer immediately before releasing that event. Workload duration includes row generation and scheduling after release. Write-batch latency measures the backend insertion call; it excludes row generation. SQL completion duration also includes the final checkpoint and intervening controller/worker-finalization time.

Deployment	Observed version / measurement boundary
SQLite native	Python 3.11.7; SQLite 3.44.2. WAL + NORMAL; automatic checkpointing at 1,000 pages and explicit final TRUNCATE checkpoint.
SQLite Shifter	Python 3.11.15; SQLite 3.46.1. Same configured checkpoint policy, different software environment.
PostgreSQL	Server 16.12; native Python clients. Automatic checkpointing plus explicit final CHECKPOINT.
InfluxDB	Server 2.9.1; native Python HTTP clients. Completion at synchronous HTTP 204 acknowledgement; no equivalent final checkpoint is timed.

PostgreSQL settings are present in the newer raw records

All 276 PostgreSQL trials in the four new archives record **fsync=on**, **synchronous_commit=on** and **full_page_writes=on**. The selected settings also show shared_buffers=16,384 blocks of 8kB (128 MiB), max_connections=100, checkpoint_timeout=300 s, max_wal_size=1,024 MB and autovacuum=on. The complete recorded selection is exported in database_configuration.csv. This is a selected settings snapshot, not a dump of every server parameter.

Statistical unit and limits

SQL repeat estimates first average five trials within each condition/allocation. Storage contrasts are paired within that allocation. Reported ratio estimates are geometric means of three allocation contrasts, with Student-t 95% intervals on their logarithms (2 degrees of freedom). These exploratory intervals assume independent, approximately normal log contrasts; no multiple-comparison adjustment is applied. The three telemetry allocations used only two distinct nodes. Mixed-pilot intervals use five within-allocation repetition contrasts and do not estimate allocation-to-allocation variation.

Counts validate stored output and expected query sizes. They do not make database durability settings or engine-specific query execution equivalent. tmpfs is volatile memory storage.

SQL writes: the repeated scaling pattern

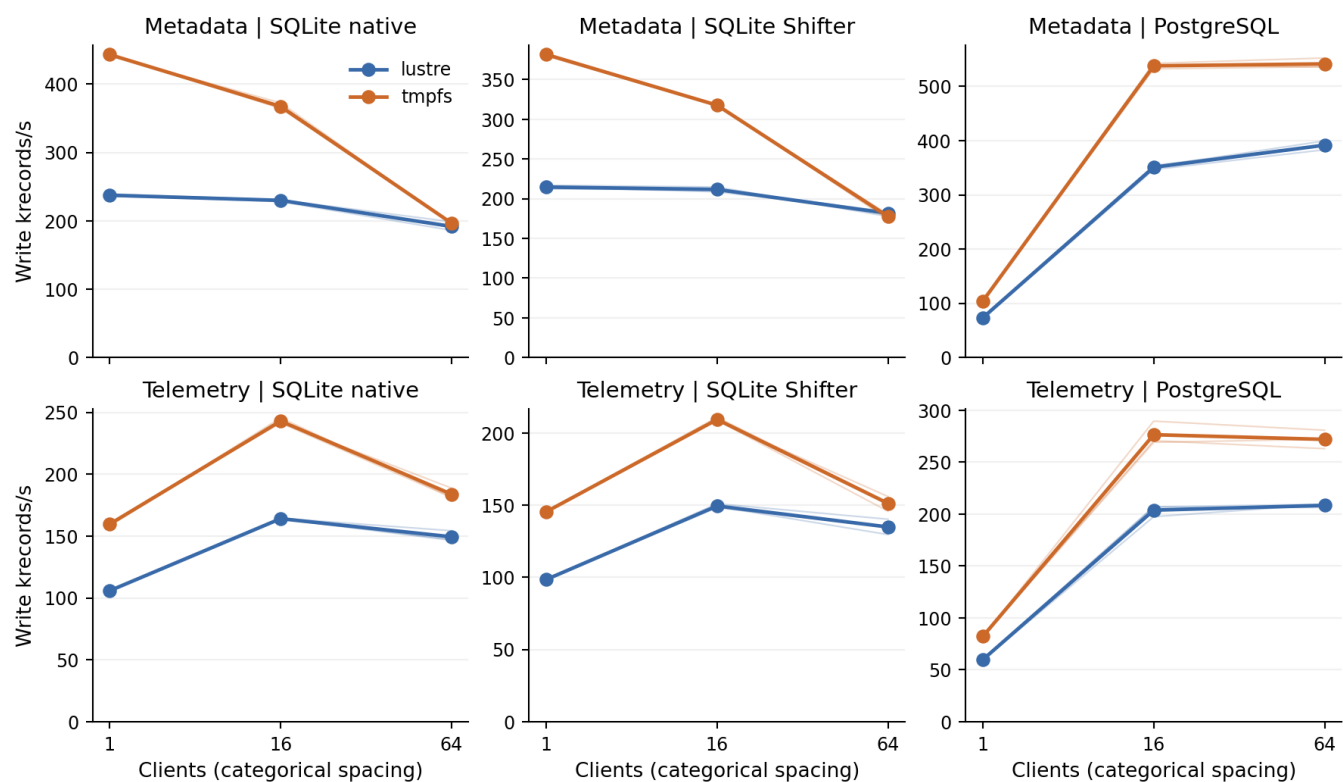


Figure 1. Bold lines: arithmetic mean of three allocation means. Faint lines: individual allocation means, each based on five measured trials. Write throughput includes the final SQL checkpoint. Each panel has its own y-axis scale.

For metadata on Lustre, PostgreSQL rises from approximately **73k to 392k records/s** between one and 64 clients. Native SQLite falls from about **237k to 192k records/s**. On tmpfs, native SQLite falls from **443k to 197k records/s**, whereas PostgreSQL rises from **104k to 542k records/s**.

Telemetry shows a different response: PostgreSQL and SQLite gain from initial concurrency, while some curves flatten or decline between 16 and 64 clients. There is no single client count that is best for all storage/database/workload combinations.

What the repetitions add

The six new jobs contain three separate allocations for metadata and three for telemetry. Every allocation runs all three SQL deployments on the same node. Deployment order rotates across blocks: native/Shifter/PostgreSQL; Shifter/PostgreSQL/native; PostgreSQL/native/Shifter. This reduces a fixed deployment-order confound, but does not eliminate time-varying shared-filesystem load or establish a universal node population effect.

Workload	Allocation jobs	Distinct compute nodes
Metadata	59304573, 59304575, 59304580	3
Telemetry	59304581, 59304582, 59304585	2 (last two allocations reused a node)

Source: sql_allocation_means.csv; all 540 measured trials remain available individually in all_trials.csv.

Storage gains and uncertainty

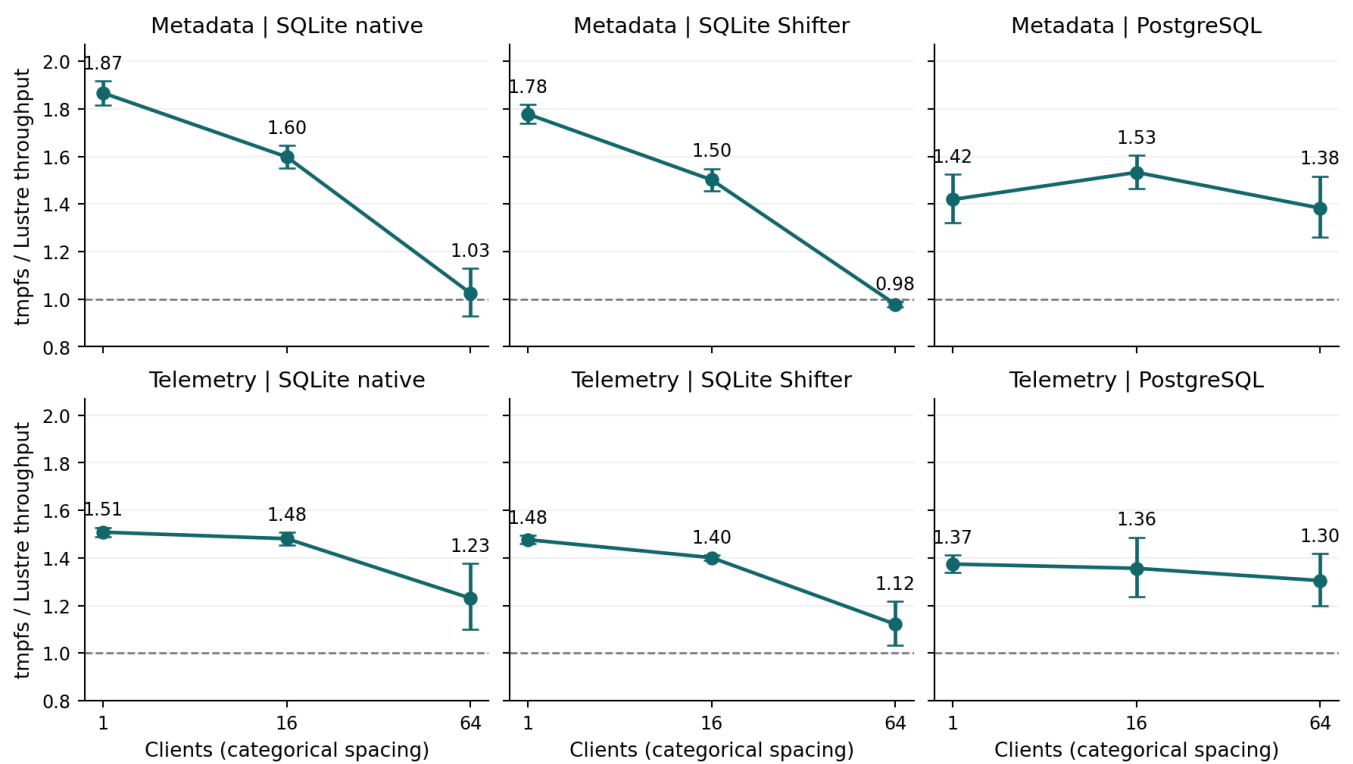


Figure 2. tmpfs/Lustre write-throughput contrasts across three allocation blocks. Whiskers are log-t 95% intervals; dashed line denotes parity. Ratios above one favor tmpfs for this measured completion boundary.

Metadata writes	1 client: ratio [95% CI]	64 clients: ratio [95% CI]
SQLite native	1.87 [1.82, 1.92]	1.03 [0.93, 1.13]
SQLite Shifter	1.78 [1.74, 1.82]	0.98 [0.97, 0.99]
PostgreSQL	1.42 [1.32, 1.53]	1.38 [1.26, 1.52]

The large single-client SQLite benefit contracts sharply at high concurrency. Native SQLite’s 64-client interval includes one. The Shifter estimate is about 2.2% below parity with a narrow interval in these three blocks; that small result remains exploratory and should not be generalized to every allocation or workload.

PostgreSQL retains a clear tmpfs advantage for metadata writes in these observations. The supported inference is an interaction between storage placement, concurrency and database behavior. Direct SQLite lock-wait timing, partitioning and busy-handler interventions were not measured here, so they cannot yet explain the cause.

Reads and execution environments

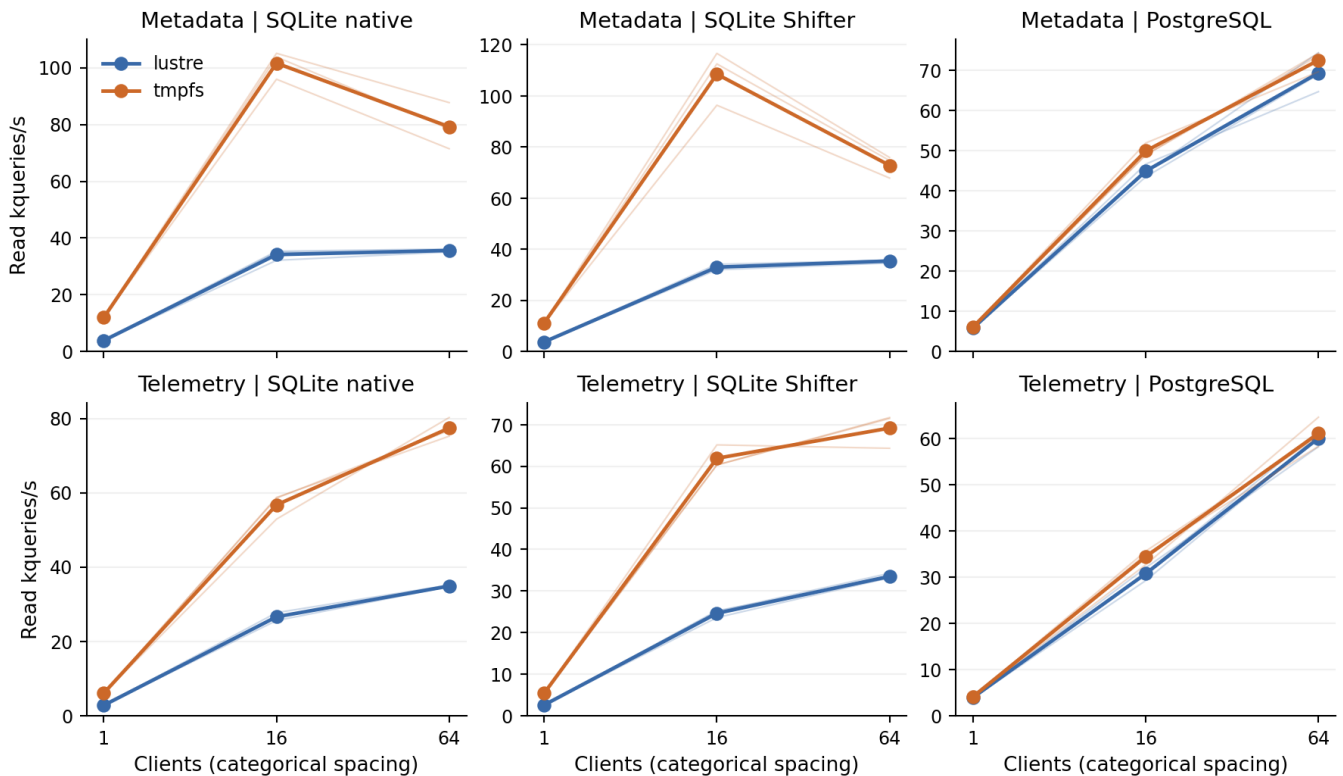


Figure 3. Warm-read throughput in the SQL allocation repeats, with 1,024 total queries per trial. Bold and faint lines use the same aggregation as Figure 1. Each panel has its own y-axis scale.

360 of 540 measured SQL-repeat read phases last under 50 ms. At 64 clients, a 1,024-query trial assigns only 16 queries to each worker. These are synchronized warm-query bursts, not evidence of sustained service throughput. Event-release skew, scheduling, caching, validation and client-side work can matter even though process startup is outside the timer.

The earlier follow-up likewise has 120 of 180 measured read phases under 50 ms. Earlier 64,000-query pilot results remain separate contextual evidence; query count, source revisions and allocations differ. InfluxDB query rates must also be interpreted in the context of its HTTP/Flux path and schema mapping.

Native versus Shifter: what is controlled

Within the allocation-repeat study, SQLite Shifter/native write ratios range from **0.82 to 0.95** across conditions. Same-node execution and rotated order improve the comparison, but Python and SQLite versions differ. These are execution-environment ratios; they do not isolate container overhead. The supplemental figure and `sqlite_environment_ratios.csv` retain all intervals, including reads.

A matched-software comparison and distributed multi-node scaling were not performed. Separate allocations on different nodes are repeated single-node experiments, not one distributed experiment.

Mixed workloads: composition matters

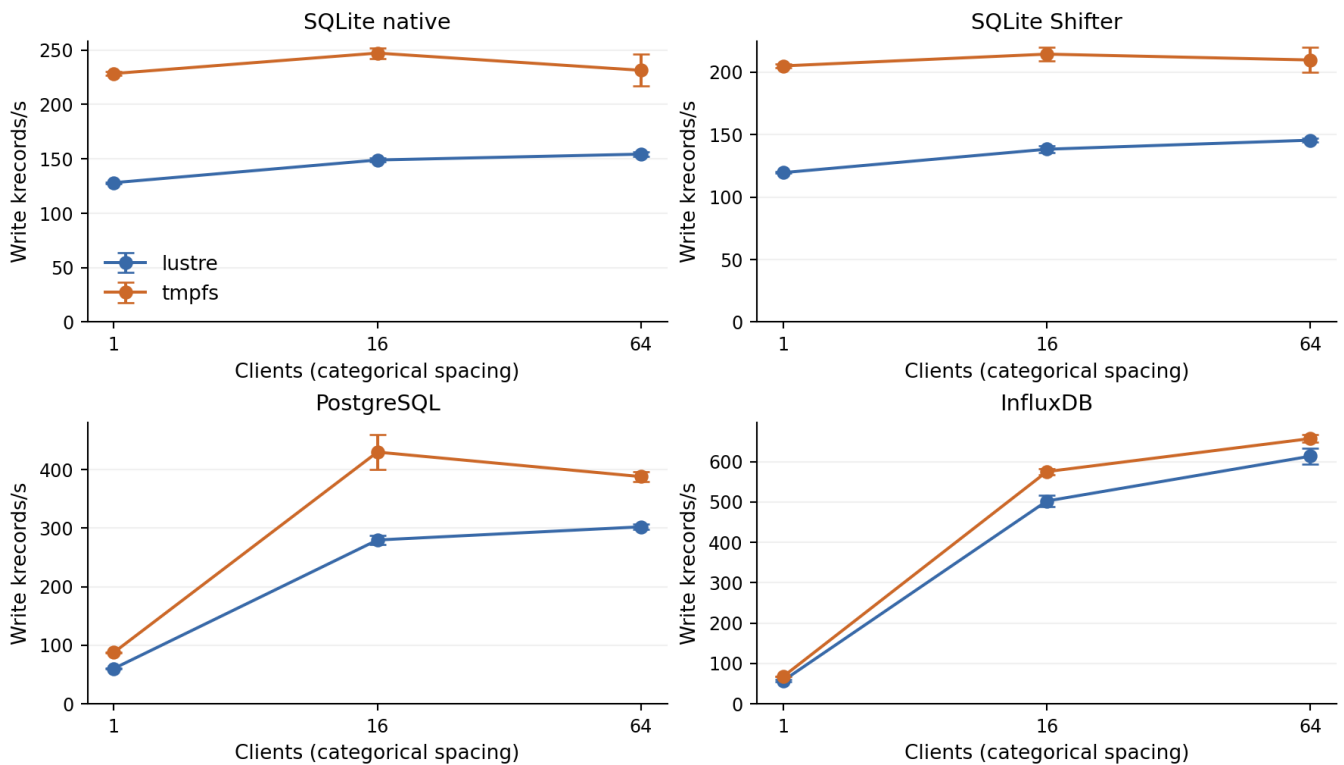


Figure 4. Mixed-workload write throughput: mean $\pm$ sample SD of five measured trials within one allocation per deployment. Y-axis scales differ. SQL includes the final checkpoint; InfluxDB ends at acknowledgement, so engine rankings are not durability-normalized.

Each trial writes 500,000 metadata and 500,000 telemetry records. Each logical batch contains two backend write calls; the pair is not one cross-type atomic transaction. Writes are followed by 512 metadata and 512 telemetry range queries. This is a mixed data-type workload, not simultaneous readers and writers.

tmpfs/Lustre mean write ratio	1 client	16 clients	64 clients
SQLite native	1.78×	1.66×	1.50×
SQLite Shifter	1.71×	1.55×	1.44×
PostgreSQL	1.46×	1.54×	1.28×
InfluxDB	1.18×	1.15×	1.07×

Native SQLite retains about a 1.50 $\times$ mean tmpfs benefit at 64 clients here, compared with near parity for metadata-only writes in the separate allocation-repeat study. Workload and campaign both differ, so this is evidence against treating the metadata-only pattern as universal, not an isolated causal composition experiment.

Tail latency remains consequential: native SQLite's median trial write p99 rises from about 28 ms to 3,581 ms on Lustre between one and 64 clients. These are logical-batch latencies, not per-record latencies.

Workload size: completed coverage and the gap

Size in this implementation	Completed evidence
100,000 records	Repeated configurations prepared; sweep was not submitted successfully.
1,000,000 records	Repeated pilots cover all four deployments, all three workloads, both tiers and 1/16/64 clients.
10,000,000 records	48 calibration observations: four deployments × three workloads × two tiers × 1/64 clients. One observation per condition; no warmups.

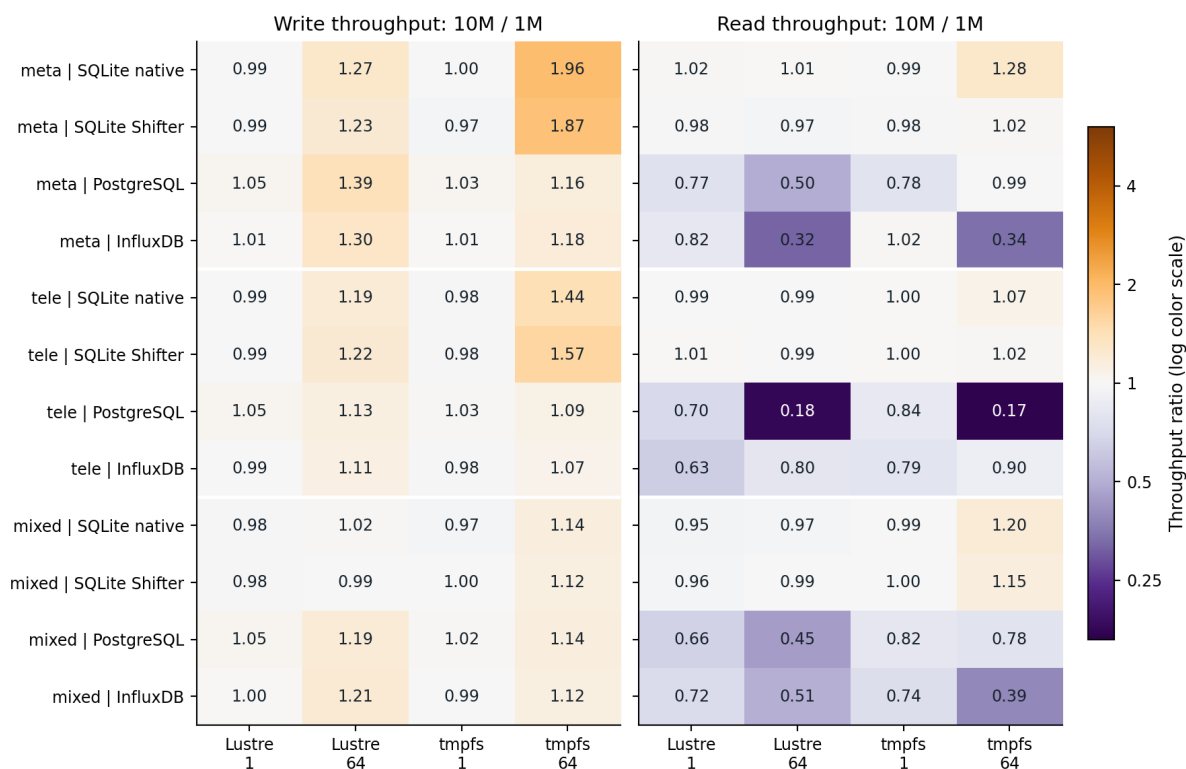


Figure 5. Descriptive 10M/1M throughput ratios. Each cell uses one 10M observation. 1M references are SQL allocation means, mixed pilots, or original InfluxDB pilots as appropriate. Separate campaigns, allocations and some source versions differ. No size-effect interval or causal scaling estimate is claimed.

The professor’s low/medium/high values were implemented as records written, while reads remain at 1,024 queries. They are not total operation counts. The 24-task repeated 100K/10M array was rejected before an array ID was assigned: estimated requested cost was 72 node-hours versus a displayed 7.00-hour balance (repository balance 6.25). This was a reservation estimate, not 72 hours consumed. Those sweeps were then explicitly omitted from this round.

System diagnostics: collected and unavailable



Figure 6. All 48 conditions from job 59329430, one observation per condition. Node-wide percentages cover the complete trial lifecycle, including setup, validation, reads, shutdown and monitors. Heatmaps use different scales; values are rounded. Row labels: meta = metadata; tele = telemetry.

The job completed in **14 minutes 46 seconds**. All node sample summaries were recomputed successfully. Node busy fractions range from 0.35% to 10.36% when averaged across the node’s 256 logical CPUs and the full lifecycle. This is not a phase-specific database CPU utilization measurement.

Sampled node used memory is MemTotal minus MemAvailable; it includes OS, cache, other node activity and tmpfs data. It is not database PSS or an isolated database memory footprint. GNU time provides waited-process CPU, maximum single-process RSS and context-switch counters. iostat logs are retained; their device counters are not treated as Lustre network/filesystem bandwidth.

Node-wide perf events were unavailable in all 48 trials. The probes were denied under perf_event_paranoid=2. Cycles, instructions, cache misses, perf context-switch counts and derived IPC therefore remain missing, not zero. No permissions were changed or additional jobs launched for the blocked probes.

Low node iowait does not rule out Lustre delays or SQLite lock waits. These diagnostics describe observed resource behavior; they do not identify the cause of the SQLite throughput collapse. Instrumentation overhead was not isolated in this diagnostic study.

Study closeout and reproducible artifact

What this round establishes

A working benchmark framework now couples storage placement, database adapters, concurrent workloads, lifecycle control, correctness checks and provenance. The data demonstrate reproducible performance patterns and show why storage choice should be evaluated together with workload and concurrency. Allocation repeats strengthen the SQL evidence; resource diagnostics provide additional context.

The contribution is a validated measurement study and reusable experimental artifact. Production readiness, publication acceptance and a proven lock-contention mechanism are not established by these results. The report is intended to support discussion and the researcher’s own manuscript preparation.

Fixed item	Closeout status
1. Mixed-workload pilots	Complete and analyzed for the implemented mixed data-type workload.
2. Workload-size coverage	1M repeated evidence and 10M calibration complete. Repeated 100K/10M sweeps explicitly excluded after budget rejection.
3. Comparisons and repeats	SQL allocation repeats validated and analyzed. Matched-software runtime comparison excluded from claims; no repeated InfluxDB allocation study.
4. System measurements	48 diagnostics validated and analyzed. Available measurements retained; blocked perf events documented.
5. Consolidated deliverables	This report, tables, figures, reproducible scripts and original inputs complete the analysis package. Publication manuscript remains the researcher’s work.

Verification and reproduction

The six input archives reconcile to 96 runs. For 1,104 trials with full raw JSON, write/read latency summaries were recomputed from saved samples. The 288 original baseline records were checked against CSV and saved metadata; that input does not contain the original per-request arrays. All 82 newly exported run source manifests match bundled source snapshots. Databases were not reopened during this offline analysis.

The companion ZIP contains 15 CSV tables, seven figures in PNG and vector PDF, original inputs, validation.json, dependency pins, SHA256SUMS and the analysis/report scripts. README.md documents reproduction and metric scope. This package adds analysis; it does not submit or rerun experiments.

Unperformed work includes distributed multi-node scaling, the professor’s partitioned-SQLite/direct-lock-wait/busy-handler experiments, matched software versions, and repeated low/high-size sweeps. Those are explicit boundaries, not silently completed checklist items.